
Autonomous Systems and Mobile Robots

MiniLab 1 – Summer Term 2026

Issued: 20.05.2026 **Due:** 10.06.2026, 23:59 **Points:** 40 (pass at 20)

Professor

Prof. Dr. Malte Schilling
mschilling@uni-muenster.de

Tutor

Janosch Bajorath
j.bajorath@uni-muenster.de

Your robot can act. It can see. Now make it react.

Over the past two weeks you built a differential-drive robot from scratch and equipped it with a 2D LiDAR sensor. In this MiniLab you close the perceive–reason–act loop: implement a ROS 2 node that reads sensor data and drives the robot to a goal—without a map and without a planner.

The provided simulation world is a walled arena with three obstacles between the robot's spawn and the goal. You *do* know where the goal is—`goal_checker_node` owns the coordinate and publishes it on the latched topic `/goal_point`—but you do *not* have a map. The task is *reactive* navigation: at each instant your node sees only the current LiDAR scan and the current odometry pose, and must steer toward the goal while avoiding whatever obstacles appear. `goal_checker_node` also signals arrival on `/goal_reached`, latched.

You work in your pre-assigned group of 2–3 students.

Submission: Submit a ZIP of your entire ROS2 workspace to LearnWeb by **10.06.2026**, 23:59. Name the file `minilab1_lastname1_lastname2.zip` (append a third name if applicable). The archive must contain all packages needed to build and run your solution from scratch with `colcon build`.

Validity rule. Your nav node's velocity commands must be driven by LiDAR data, and motion must go through the `SetVelocity` action client — your node does not publish `/cmd_vel` directly. A submission that ignores the LiDAR or bypasses the action client is rejected regardless of whether the robot arrives.

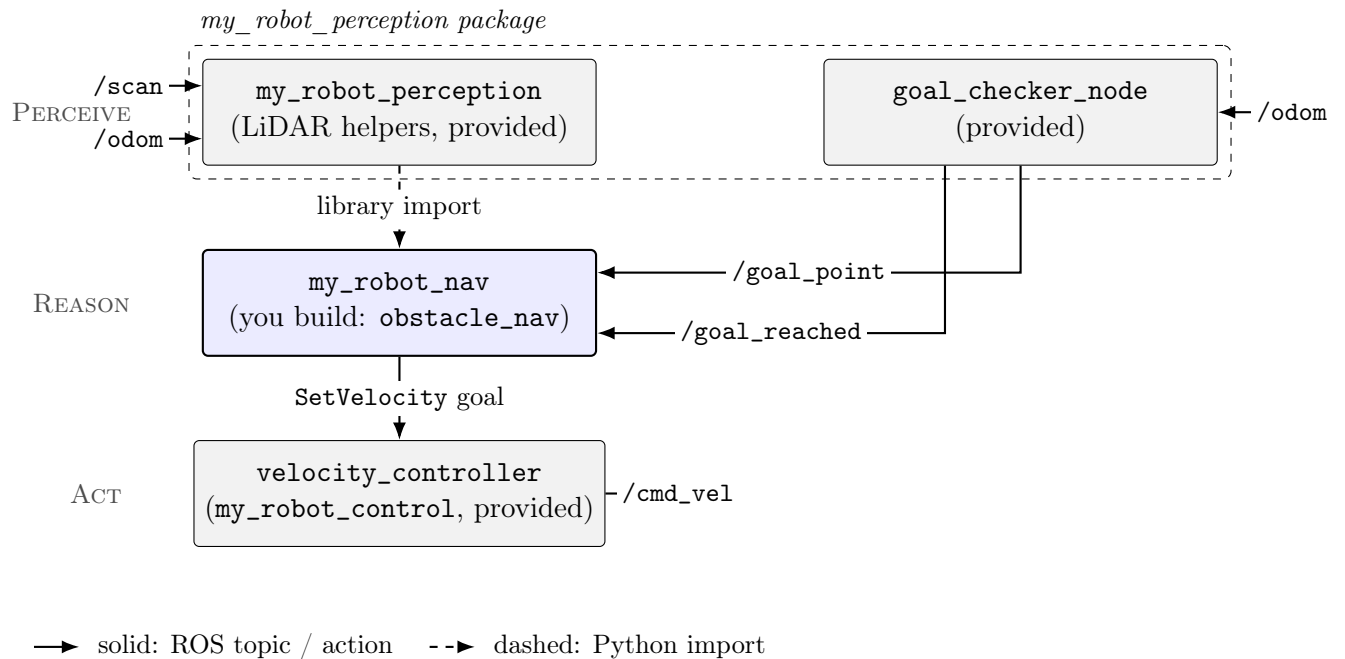
Starting Point

Your workspace contains everything you need to start. The table below groups every package and file by where it comes from — code you wrote in earlier weeks, code provided in this week's starter zip, and the two files you write yourself.

Package / file	Role
<i>From your previous weeks (already in your workspace)</i>	
my_robot_description	URDF: body, wheels, LiDAR.
my_robot_bringup	Launch files, <code>bridge.yaml</code> , RViz config, world SDFs.
my_robot_control	Open-loop motion helpers (your code from Week 2).
<i>Provided this week (in <code>reference_ws/</code>)</i>	
obstacle_world.sdf	Gazebo world for this minilab; drop into <code>my_robot_bringup/worlds/</code> .
velocity_controller_node	Added into <code>my_robot_control</code> : hosts the <code>SetVelocity</code> action server; the only node that publishes <code>/cmd_vel</code> during this minilab.
my_robot_perception	Sensor helpers: <code>sector_min</code> , <code>get_position</code> , <code>get_yaw</code> , <code>pointcloud2_to_xy</code> . <code>goal_checker_node</code> : owns the goal coordinate, publishes <code>/goal_point</code> and <code>/goal_reached</code> . <code>scan_to_pointcloud_node</code> (opt-in): RViz-friendly cartesian view of the scan.
my_robot_interfaces	<code>ament_cmake</code> package defining <code>SetVelocity.action</code> . You will write a similar interface package from scratch in MiniLab 2.
<i>You will build</i>	
my_robot_nav/obstacle_nav.py	The reactive nav node.
my_robot_bringup/launch/sim.launch.py	Brings up the obstacle world plus the provided nodes.

Architecture

Every robot in this course is organised around the **Perceive–Reason–Act** loop. In MiniLab 1, that loop maps onto three packages. `my_robot_perception` is the **Perceive** layer: LiDAR helpers and `goal_checker_node`. `my_robot_nav` is the **Reason** layer, the reactive node *you* build. `velocity_controller_node` in `my_robot_control` is the **Act** layer, turning velocity goals into `/cmd_vel`. The diagram below shows the wiring.



The dashed arrow is a library import, not a ROS topic: `my_robot_perception` provides helper functions (`sector_min`, `get_position`, `get_yaw`) that your nav node calls directly. The solid arrows are real topics or action interfaces on the ROS graph.

Notice that your nav node never publishes on the `/cmd_vel` topic itself. During this minilab, `velocity_controller_node` in `my_robot_control` owns that publisher, and your nav node sends `SetVelocity` action goals to it. The `/cmd_vel` stream feeds Gazebo's `diff_drive_controller` exactly as in Week 2.

The `goal_checker_node` owns the goal coordinate. It publishes it on `/goal_point` (a latched `geometry_msgs/PointStamped`) so your nav node can subscribe and steer toward it—this is the single source of truth for “where the goal is.” The `/goal_reached` arrow is the matching event signal: the goal-checker watches odometry against the same coordinate and announces arrival once. You can wire your nav node to subscribe and stop moving on arrival.

Task 1 – Reactive Obstacle Navigation

(40 pts)

The obstacle world places three obstacles (one box and two cylinders) between the robot's spawn and the goal. You know the goal coordinate (published on `/goal_point`) but you do not have a map, and the obstacles sit on or near the straight line between spawn and goal. A constant-velocity Twist toward the goal would collide. In this task you build the `my_robot_nav` package, write the simulation launch file, and implement a reactive node that perceives the obstacles and steers through them.

The two files you add to your workspace:

```
my_robot_bringup/
...                # existing contents from Week 2 + 3
launch/
...                # your existing launch files
sim.launch.py      # you build

my_robot_nav/      # you build
package.xml
setup.py
my_robot_nav/
__init__.py
obstacle_nav.py    # you build
```

- (a) **Create the `my_robot_nav` package and write `obstacle_nav.py`.** An ament_python package:
- `package.xml` declaring the dependencies your nav node needs (`roslpy`, `sensor_msgs`, `nav_msgs`, `std_msgs`, `geometry_msgs`, `my_robot_interfaces`, `my_robot_perception`).
 - `setup.py` registering one console-script entry point: `obstacle_nav`.
 - `my_robot_nav/obstacle_nav.py` implementing the reactive node. It must:
 - Subscribe to `/goal_point` (`geometry_msgs/PointStamped`, latched QoS, durability `TRANSIENT_LOCAL`) to receive the goal.
 - Subscribe to the LiDAR topics your algorithm uses (`/scan` and/or `/scan_points`) and to `/odom` for the robot pose.
 - Use a `SetVelocity` action client to command velocities. The action is defined in `my_robot_interfaces`; the server lives in `velocity_controller_node` (started by your launch file in part (b)). *Your node does not publish `/cmd_vel` directly.*
 - Publish or log at least one named visualization / debugging signal — e.g. an RViz marker for the chosen heading, a throttled log of a key internal variable, or a `geometry_msgs` arrow. The signal must be observable while the node runs and named in your submission's `README` (one sentence: what it is, how to view it).

- Subscribe to `/goal_reached` (latched QoS, durability `TRANSIENT_LOCAL`). When the value becomes `true`, send a final `SetVelocity` goal with zero velocity to stop cleanly.

There is no prescribed algorithm. A potential field is one natural choice for an advanced robotics engineer (attractive force toward the goal, repulsive force away from obstacles); a sector-based “avoid front, steer toward goal” rule works fine too. The helpers in `my_robot_perception` (`sector_min`, `get_position`, `get_yaw`, `pointcloud2_to_xy`) are there for you to use. An opt-in `scan_to_pointcloud_node` is also available if you prefer working with cartesian points in `base_link`. Feel free to write your own helper functions as needed.

(b) **Write the simulation launch file** `my_robot_bringup/launch/sim.launch.py`. It must:

- Load the obstacle world SDF from `my_robot_bringup/worlds/` and spawn the robot at the position documented in the world-file header comment.
- Start `robot_state_publisher` and `ros_gz_bridge` (reuse `my_robot_bringup/config/bridge.yaml`).
- Start the provided `goal_checker_node` from `my_robot_perception`, passing `goal_x` and `goal_y` parameters matching the world (also documented in the world-file header).
- Start the provided `velocity_controller_node` from `my_robot_control`. This is the action server your nav node uses to issue velocity commands.

(c) **Build, run, and verify.**

Build the workspace, source it, launch the obstacle world via `ros2 launch my_robot_bringup sim.launch.py`, and run your node with `ros2 run my_robot_nav obstacle_nav`. Confirm the robot reaches the goal region — you will see `GOAL REACHED` from `goal_checker_node` in the launch terminal, and the robot stops.

Refresher pointers. Each technique you need was covered in a previous exercise. If something feels unfamiliar:

- Creating a Python ROS2 package — Week 2, Task 1
- Writing a subscriber node, `setup.py` entry point, `colcon build` with `--symlink-install` — Week 3, Task 6
- Writing an action client — Week 1, Task 7 (`turtle_star`)
- Computing the forward-facing index of `/scan` — Week 3, Task 4
- Launch files with declared arguments — Week 2 (visualisation launch file)

Grading:

*A submission that violates the validity rule (ignores LiDAR or bypasses the **SetVelocity** action client) receives 0 points regardless of individual bullet scores.*

- `my_robot_nav` package proper: builds cleanly, `package.xml` declares the dependencies, `setup.py` registers the `obstacle_nav` entry point 5 pts
- `sim.launch.py` brings up the obstacle world without errors; robot spawns at the documented position; `goal_checker_node` and `velocity_controller_node` log their startup messages 5 pts
- Correctly wired: subscribes `/goal_point`, `/scan` (and/or `/scan_points`), and `/odom`; commands motion only via the **SetVelocity** action client (no direct `/cmd_vel`) 5 pts
- Reactive behavior: steering visibly depends on LiDAR data, per the validity rule 5 pts
- Visualization / debugging signal implemented and observable when the node runs; named in the submission **README** 5 pts
- No visible collisions during the run 5 pts
- Robot reaches the goal region; clean stop on `/goal_reached` (final zero-velocity goal sent on arrival) 10 pts